

Tejas Kabra

Electrical Engineering Portfolio

Texas A&M University - Embedded Systems, PCB Design, Power Electronics, Robotics
Houston, TX - kabratejas24@gmail.com - linkedin.com/in/tejas-kabra-0869aa291

Portfolio contents

Formula Electric CAN-LTE Telemetry System

STM32G4, CAN, UART, LTE modem, four-layer PCB, 3,200 packets/s, 2-3 ms latency

High-Side NMOS Transient Suppression and Linear Regulator

12 V / 5 A load protection during 50 V / 50 ms input transient

Automated Electronic Inventory System

Raspberry Pi, touchscreen workflow, card authentication, MOSFET-driven solenoids

Object-Carrying Drone Platform

Airframe integration, Betaflight setup, Arduino bridge, camera and payload packaging

Focus

Hardware and embedded systems work with emphasis on schematic decisions, PCB layout, power architecture, firmware integration, test results, and revision reasoning.

Formula Electric CAN-LTE Telemetry System

Texas A&M Formula Electric - Low Voltage Electronics

STM32G4

CAN

LTE

4-layer PCB

3,200/s

TELEMETRY PACKETS

2-3 ms

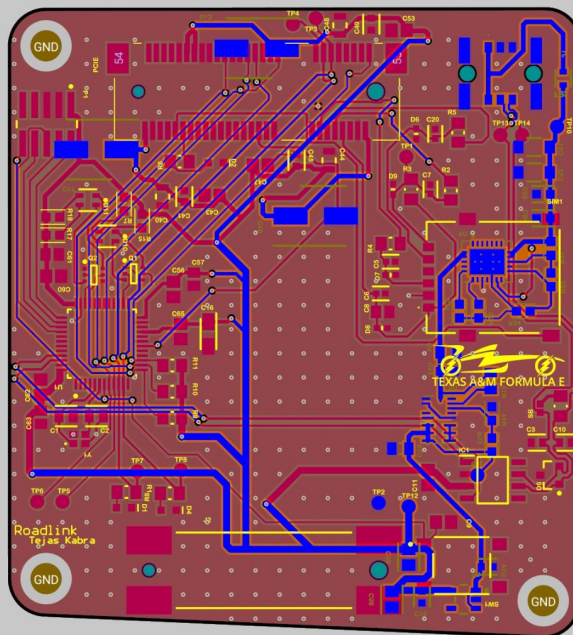
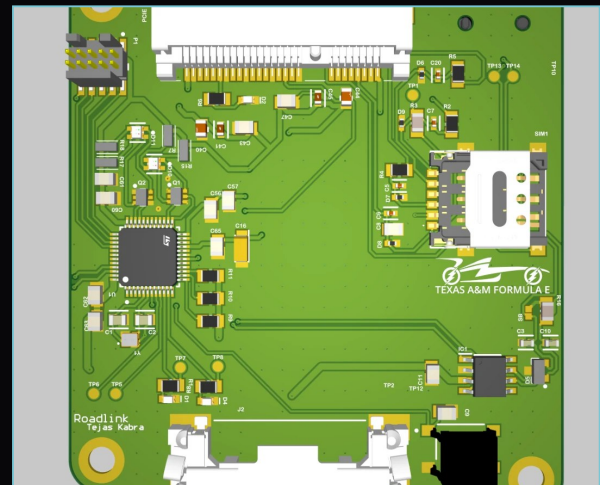
END-TO-END LATENCY

2 revs

PCB ITERATIONS

Texas A&M Formula Electric's racecar distributes critical operating data over CAN. The telemetry system moves those diagnostics off the vehicle in real time so engineers can monitor behavior from a Grafana-based driver station rather than waiting for a post-run download.

Designed the telemetry PCB across two revisions, developed its embedded firmware, and validated the integrated system before vehicle deployment. The board combines an STM32G4 microcontroller, CAN and UART interfaces, a backplane vehicle-power input, and a SIM-enabled Quectel EG25 LTE modem.



R2 front render and four-layer layout. The board routes CAN, UART, LTE, SIM, protected power, buck regulation, LDO regulation, and local filtering.

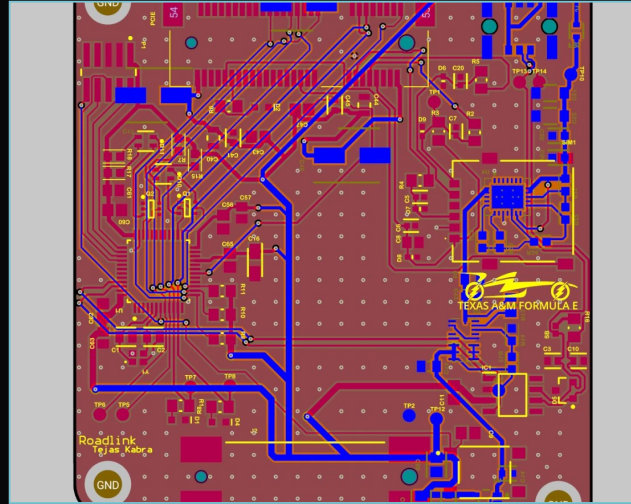
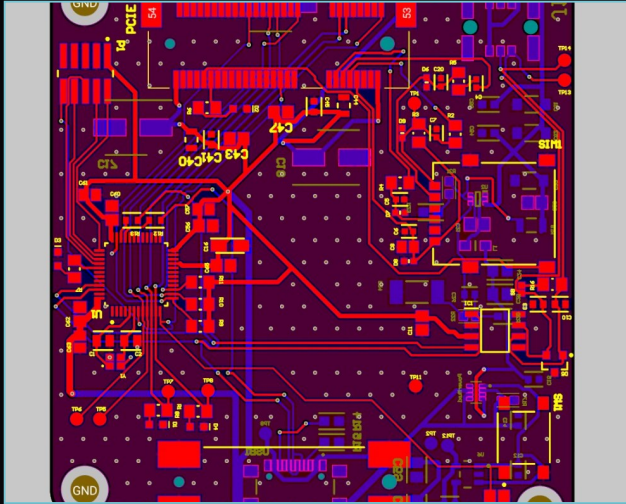
Telemetry: Revision and Firmware Detail

Roadlink R1 to R2 and CAN-to-LTE data path

eFuse

Buck regulation

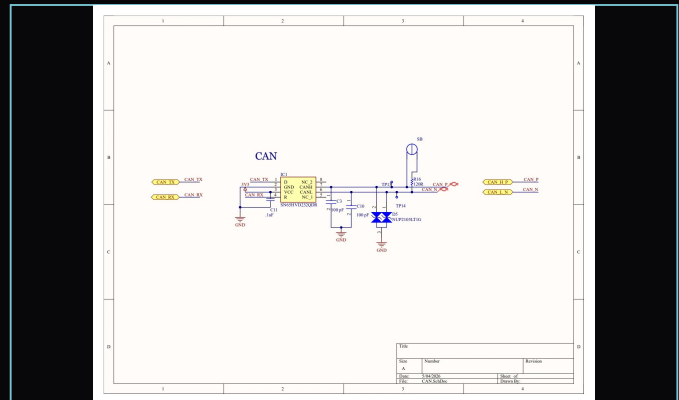
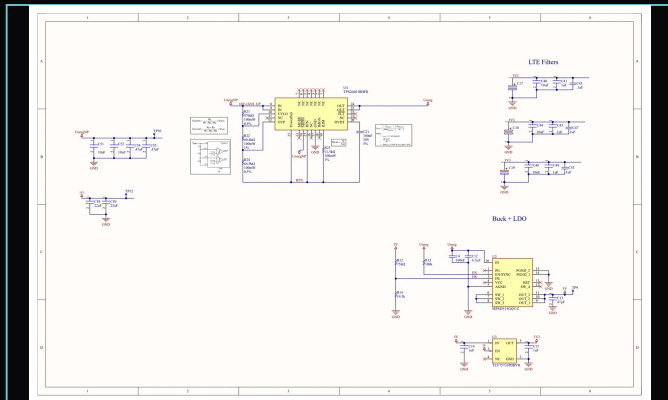
STM32 HAL



R1 to R2

R1 proved the basic system concept and clarified the mechanical constraints, connector placement, STM32 routing, LTE interface, SIM section, and CAN path. R2 is the more intentional revision, with a cleaner layout and a more defensible electrical architecture.

- R2 replaces the weaker first buck approach with a more stable buck architecture and added support passives.
- R2 adds an eFuse so the input path handles fuse behavior, undervoltage, and overvoltage protection.
- R2 moves to four layers with reference planes between UART and CAN signals for cleaner communication paths.
- Firmware in C using STM32CubeIDE and HAL acquires CAN frames, builds 8-byte telemetry payloads, and transfers them to the LTE modem.



High-Side NMOS Transient Suppression

Power electronics - 12 V / 5 A load, 50 V / 50 ms transient

LTspice

Altium

OPA197

LT1054

12 V / 5 A

NOMINAL LOAD

50 V

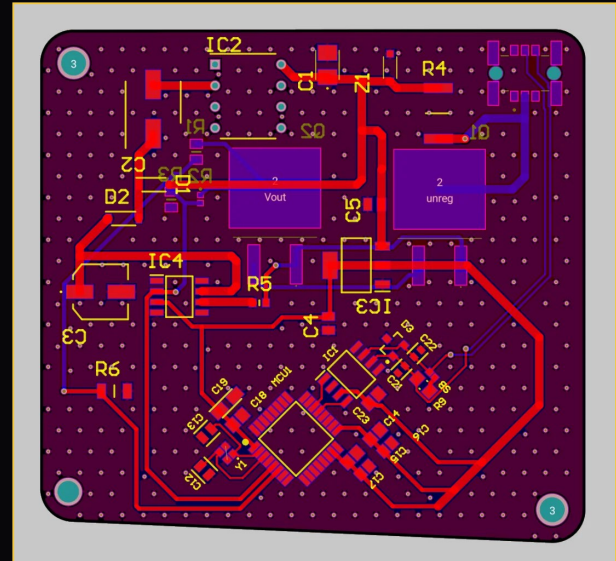
SURGE INPUT

13.8 V

SIMULATED CLAMP

This board protects a 12 V, 5 A load when the input line jumps to 50 V for 50 ms. The output cannot follow the input directly, and the pass devices, gate drive, and control electronics all have to survive the transient.

The circuit is a high-side linear protection stage. The main current path uses two back-to-back N-channel MOSFETs, while an op-amp feedback loop drives the MOSFET gates to regulate the protected output. In LTspice, when V_{in} rises to 50 V, V_{out} rises to about 13.8 V before returning to 12 V after the pulse.



Surge Board: Gate Drive, Clamp Point, Validation

Analog control loop and transient simulation

High-side NMOS

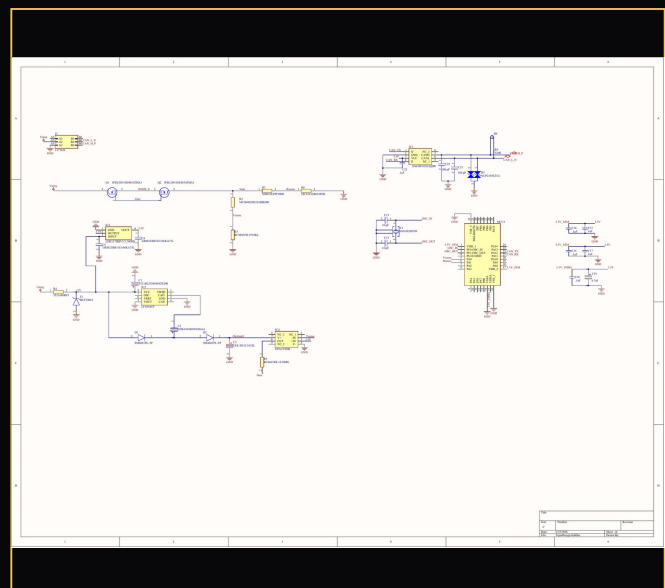
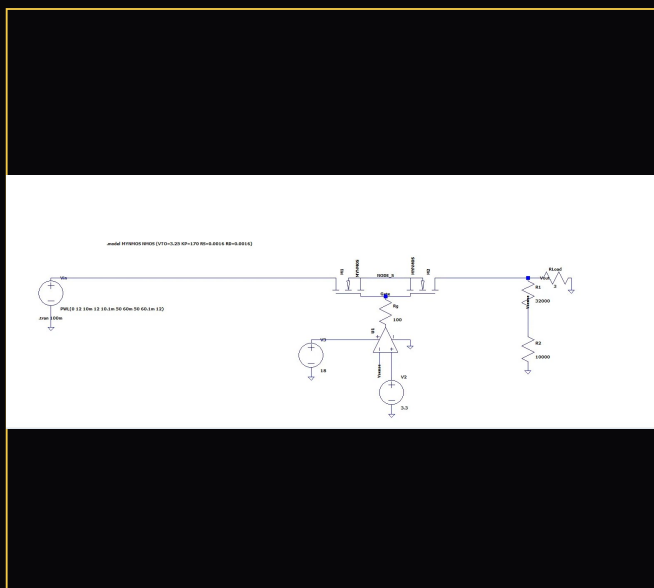
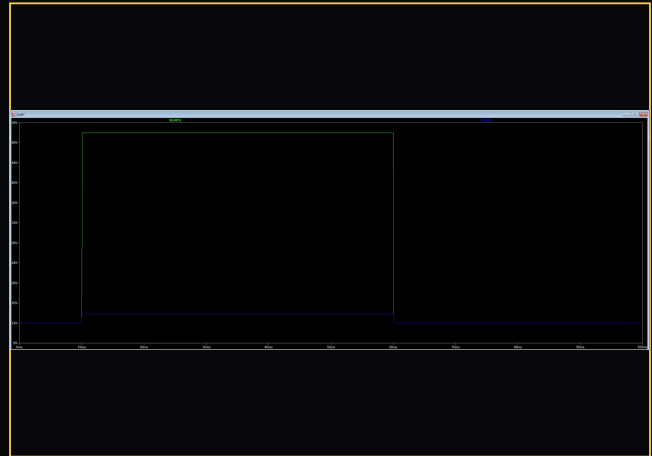
Charge pump

SOA

Because the MOSFETs sit on the high side, the source node rises near the protected output. A zener-limited 12 V control rail feeds an LT1054 charge pump so the op-amp can command a gate voltage in the roughly 18 V to 20 V range under load.

The op-amp compares the output sense voltage against the 3.3 V reference. A 32 k / 10 k divider scales the output by $10 / (32 + 10) = 0.238$, so the clamp target is $3.3 / 0.238$, or about 13.9 V.

Future correction: the first MOSFET in the high-side path can take most of the heat during the surge event. A future board would use wider traces, larger copper pours, more copper tied to the MOSFET pads, and heatsinking around the leading MOSFET.



Automated Electronic Inventory System

Formula Electric shelf access and checkout system

Raspberry Pi

Electron

Flask

MOSFET drivers

15

SOLENOID OUTPUTS

17.5 A

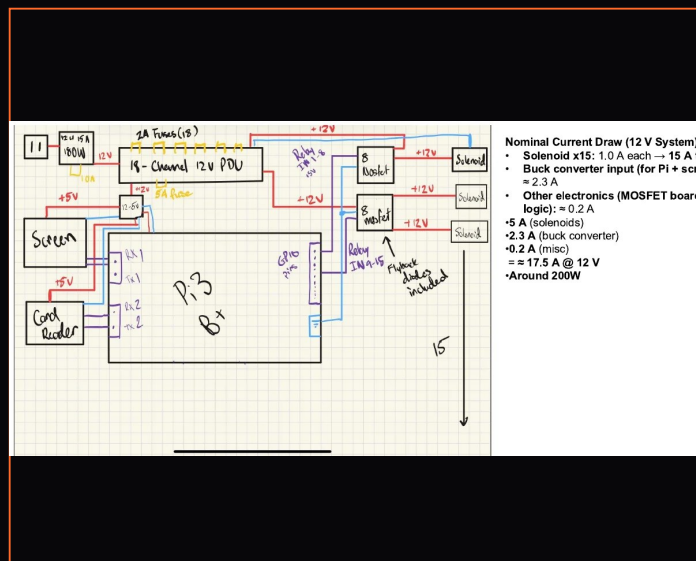
ESTIMATED 12 V DRAW

\$442

ELECTRONICS BOM TARGET

Formula Electric electrical components were organized physically, but team members still had to search bins manually and track access separately. This project connects authenticated digital checkout to physical shelf access, so a user can identify themselves, select parts, and open the correct inventory location.

The system architecture combines a Raspberry Pi controller, touchscreen interface, card reader, 12 V solenoids, Adafruit 8-channel N-MOSFET driver boards, fused power distribution, and a CAD-packaged electronics enclosure.



Item	Amount	Cost/unit	Cost(total)	Link
Pi 5 2gb	1	50	50	pi 5
Buck Converter 5pc	1	7.99	7.99	Buck Converter
Programmable Screen 7 inch	1	45.99	45.99	Brush
Solenoid(subject to change)	15	14.99	224.85	Solenoid (pull type)
8 channel n-mosfet boards	2	14.95	29.9	nmosfet
2A fuses	15	.39	5.85	fuses
PDU reference	1	48	48	PDU
Card reader	1	20.42	20.42	card reader
Wall mount 12v	1	9	9	wall mount
Total			442	

Inventory Shelf: Software, Packaging, Integration

Checkout workflow, mechanical constraints, and actuator stack

Card/UIN auth

12 V rail

Printed housings

The Raspberry Pi owns the user interface, authentication, checkout state, and hardware command logic. The software design review used an Electron front end with a Flask backend communicating to the hardware layer through PySerial or a similar serial interface.

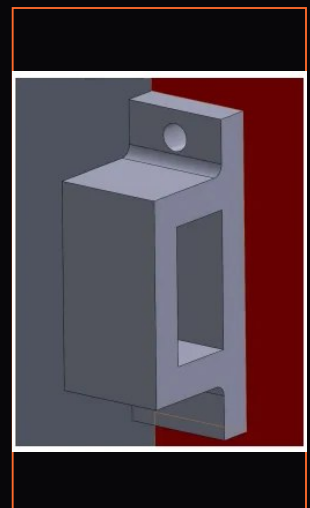
The checkout flow was scoped around swipe or UIN authentication, scoped admin permissions, item and quantity selection, withdraw and deposit modes, and drawer-by-drawer access so only one location opens at a time.

The mechanical review identified hollow side walls, drawer rail clearance, top-side fastening, aluminum locking rods, printed solenoid sheaths, and a wire pass-through for roughly 30 wires.



- Electron app + Flask backend interfacing through Pyserial (or adjacent library)

- MVP:
 - Auth through swipe and UIN, scoped admins
 - Third screen lets you select items + quantities, add to cart if withdrawing
 - Take out items drawer by drawer (one drawer opens at a time)
 - Third screen lets you select items + quantities, and add new items if depositing
 - Also define what quantifies something as "low"
 - Endpoint that exposes current items + quantities to be viewed in OMS and app for live tracking
 - Admin functionality to open all drawers at once
 - Notifications on login if something is low?
- Post MVP:
 - Board rev functionality
 - Notifications if something is low
 - TBD: Prefilled orders based on historical data



Object-Carrying Drone Platform

UAV integration, Betaflight setup, Arduino bridge, payload packaging

STM32F411

Betaflight

Arduino

\$180 BOM

This project started as a Drone Challenge UAV build for Gulf Coast Texas. The work covered the airframe, camera link, radio-control path, Betaflight setup, wiring, CAD layout, and payload rules.

The final prototype used a carbon-fiber drone base, brushless motor layout, Li-ion battery pack, camera, radio/controller hardware, an Arduino bridge, and a flight controller running Betaflight on an STM32F411 microcontroller.

The Arduino acted as a bridge between the physical controller and the flight controller. Camera integration used Arduino code with Wire.h for I2C communication, sending camera information through the Arduino and radio path to the driver's visual setup.

